

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug._2026

COURSE NAME: Artificial Intelligence- MODEL PRACTICAL EXAMINATION

COURSE CODE: CSA1709

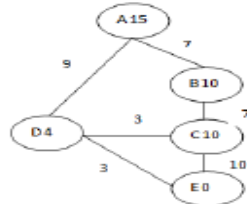
Register Number	KAVIYA.G
Name	192524281

SET:15

1. Write a Python program to implement a Map Colouring problem with the following data. Given a map of countries, and a fixed set of colours {Red, Green, Blue}, assign a colour to each region in the map in such a way that no two adjacent regions have the same colour. Solve the above Map Colouring problem using backtracking in Constraint Satisfaction Procedure.



2. Write a Python program to apply A* search for the given tree, in which 'A' is the initial state and 'E' goal state. For each node in each stage, obtain cost estimates where $g(n)$ is the numeral by the side of an arc and $h(n)$ is the numeral at the node.



3. Write a Prolog program to implement Inference Engine with the following facts
`interpret(true) :- !.`
`interpret((GoalA,GoalB)) :- !,`
`interpret(GoalA),`
`interpret(GoalB).`
4. Write a Prolog program to find the number of vowels in the following fact.
This is my first Degree in Saveetha School of Engineering.

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

1. Map Colouring Using Backtracking

AIM

To implement the **Map Colouring Problem** using **Backtracking in a Constraint Satisfaction Problem (CSP)** such that no two adjacent regions have the same colour.

ALGORITHM

1. Start with the given map regions and available colours {Red, Green, Blue}.
2. Represent the map using a **graph**, where each region is a node and adjacent regions are connected by edges.
3. Select an uncoloured region.
4. Assign one of the available colours to the selected region.
5. Check whether the assigned colour conflicts with any already coloured adjacent region.
6. If there is **no conflict**, proceed to the next uncoloured region.
7. If a conflict occurs, try the next available colour.
8. If no colour is possible, **backtrack** to the previous region and change its colour.
9. Continue until all regions are successfully coloured.
10. Display the final map-colouring solution.

PROGRAM:

```
# Map Colouring Problem using Backtracking

colors = ["Red", "Green", "Blue"]

regions = [
    "Western Australia",
    "Northern Territory",
    "South Australia",
    "Queensland",
    "New South Wales",
    "Victoria",
    "Tasmania"
]

# Adjacent regions
graph = {
    "Western Australia": ["Northern Territory", "South Australia"],
    "Northern Territory": [
        "Western Australia", "South Australia", "Queensland"
    ],
    "South Australia": [
        "Western Australia", "Northern Territory",
        "Queensland", "New South Wales", "Victoria"
    ],
    "Queensland": [
        "Northern Territory", "South Australia", "New South Wales"
    ],
    "New South Wales": [
        "Queensland", "South Australia", "Victoria"
    ],
    "Victoria": [
        "South Australia", "New South Wales"
    ],
    "Tasmania": []
}

# Check whether the colour is safe
def is_safe(region, color, assignment):
    for neighbour in graph[region]:
        if neighbour in assignment and assignment[neighbour] == color:
            return False
    return True

# Backtracking function
def backtracking(assignment):
```

```

# Backtracking function
def backtracking(assignment):
    if len(assignment) == len(regions):
        return True

    # Select an uncoloured region
    for region in regions:
        if region not in assignment:

            # Try each colour
            for color in colors:
                if is_safe(region, color, assignment):
                    assignment[region] = color

                    # Recursive call
                    if backtracking(assignment):
                        return True

            # Backtrack
            del assignment[region]

    return False

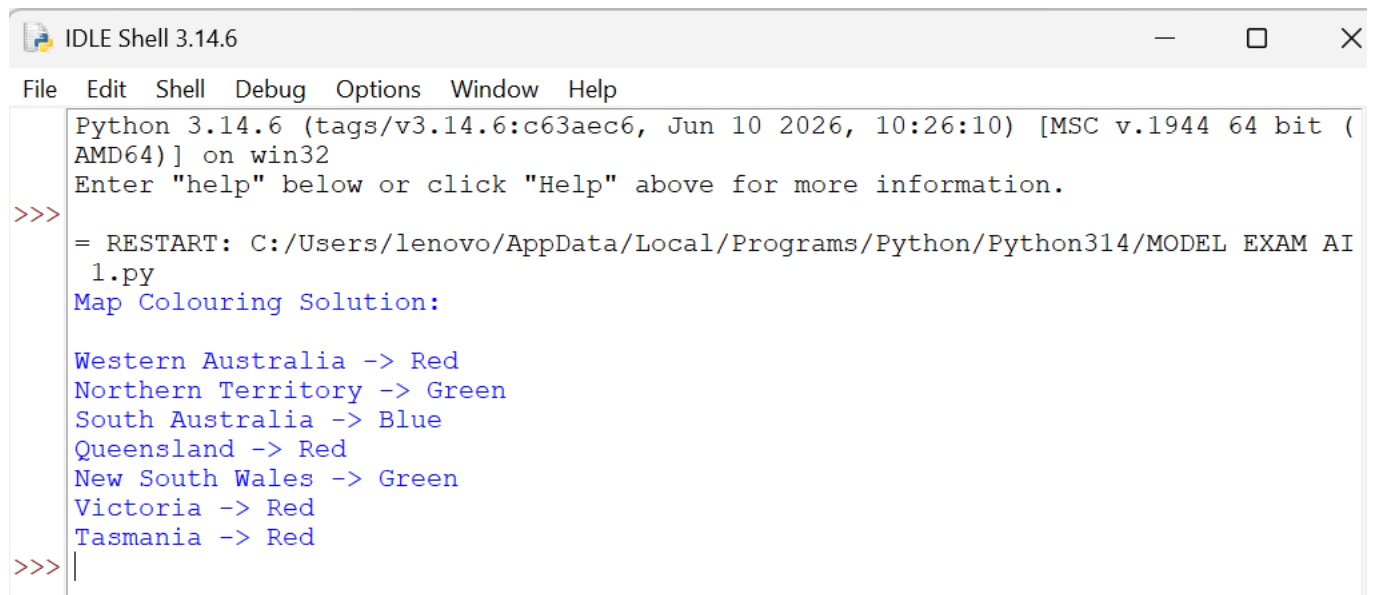
# Main program
assignment = {}

if backtracking(assignment):
    print("Map Colouring Solution:\n")

    for region in regions:
        print(region, "->", assignment[region])
else:
    print("No solution exists.")

```

OUTPUT:



```

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/MODEL EXAM AI 1.py
Map Colouring Solution:

Western Australia -> Red
Northern Territory -> Green
South Australia -> Blue
Queensland -> Red
New South Wales -> Green
Victoria -> Red
Tasmania -> Red
>>>

```

RESULT

Thus, the **Map Colouring Problem** was successfully implemented using **Backtracking in CSP**, ensuring that no two adjacent regions have the same colour.

2. A* SEARCH ALGORITHM

AIM

To implement the **A* Search Algorithm** to find the optimal path from the initial state **A** to the goal state **E** using the given edge costs and heuristic values.

ALGORITHM

1. Start with the initial node **A** and set its path cost $g(n)$ to 0.
2. Calculate the evaluation function using:
 $f(n) = g(n) + h(n)$.
3. Insert the initial node into the **OPEN list**.
4. Select the node having the **lowest $f(n)$** value.
5. If the selected node is the goal node **E**, stop and return the path.
6. Otherwise, expand the selected node and generate its neighbouring nodes.
7. Calculate $g(n)$, $h(n)$, and $f(n)$ for each neighbouring node.
8. Add the unvisited neighbouring nodes to the OPEN list.
9. Continue selecting the node with the smallest $f(n)$ value.
10. Repeat until the goal node **E** is reached.
11. Display the optimal path and its total cost.

PROGRAM:

```
# A* Search Algorithm

import heapq

# Graph with edge costs
graph = {
    'A': {'B': 7, 'D': 9},
    'B': {'A': 7, 'C': 7},
    'D': {'A': 9, 'C': 3, 'E': 3},
    'C': {'B': 7, 'D': 3, 'E': 10},
    'E': {'D': 3, 'C': 10}
}

# Heuristic values h(n)
heuristic = {
    'A': 15,
    'B': 10,
    'C': 10,
    'D': 4,
    'E': 0
}

def a_star(start, goal):
    # Priority queue: (f, g, node, path)
    open_list = [
        (heuristic[start], 0, start, [start])
    ]

    # Store minimum cost to reach each node
    g_cost = {start: 0}

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        # Goal reached
        if current == goal:
            return path, g

        # Explore neighbours
        for neighbour, cost in graph[current].items():
            new_g = g + cost

            # If a better path is found
```

```

def a_star(start, goal):

    # Priority queue: (f, g, node, path)
    open_list = [
        (heuristic[start], 0, start, [start])
    ]

    # Store minimum cost to reach each node
    g_cost = {start: 0}

    while open_list:

        f, g, current, path = heapq.heappop(open_list)

        # Goal reached
        if current == goal:
            return path, g

        # Explore neighbours
        for neighbour, cost in graph[current].items():

            new_g = g + cost

            # If a better path is found
            if neighbour not in g_cost or new_g < g_cost[neighbour]:

                g_cost[neighbour] = new_g

                # f(n) = g(n) + h(n)
                new_f = new_g + heuristic[neighbour]

                heapq.heappush(
                    open_list,
                    (new_f, new_g, neighbour, path + [neighbour])
                )

    return None, float('inf')

# Main program
path, cost = a_star('A', 'E')

print("A* Search Path:")
print(" -> ".join(path))

print("Total Cost:", cost)

```

OUTPUT:

```

LOCAL QUESTION 2.py
A* Search Path:
A -> D -> E
Total Cost: 12

```

RESULT

Thus, the **A* Search Algorithm** was successfully implemented, and the optimal path from **A to E** is: **A → D → E** with a minimum total cost of **12**.

3. PROLOG INFERENCE ENGINE

AIM

To implement an **Inference Engine in Prolog** using facts, rules, and the **cut (!) operator** to process different goals.

ALGORITHM

1. Start the Prolog program.
2. Define the interpret predicate.
3. Define a rule to handle a list containing two goals.
4. Define rules to handle individual goals.
5. Use the **cut operator (!)** to prevent unnecessary backtracking.
6. When a goal is given, match it with the appropriate interpret rule.
7. Display the corresponding interpretation.
8. Execute the query in the Prolog environment.
9. Display the result.

PROGRAM :

```
% Inference Engine

interpret :- !.

interpret([GoalA, GoalB]) :-
    !,
    write('Goal A: '),
    write(GoalA),
    nl,
    write('Goal B: '),
    write(GoalB),
    nl.

interpret(GoalA) :-
    !,
    write('Goal: '),
    write(GoalA),
    nl.
```

OUTPUT:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- ['C:/Users/lenovo/OneDrive/Documents/QUES 3.pl'].
compiling C:/Users/lenovo/OneDrive/Documents/QUES 3.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/QUES 3.pl compiled, 17 lines read - 1097 bytes written, 7 ms

(31 ms) yes
| ?- interpret(goal_a).
Goal: goal_a

yes
| ?- interpret((goal_a, goal_b)).
Goal: goal_a,goal_b

yes
| ?- |
```

RESULT

Thus, the **Prolog Inference Engine** was successfully implemented using **rules and the cut (!) operator** to control backtracking.

4. PROLOG – COUNT NUMBER OF VOWELS

AIM

To write a **Prolog program to find the number of vowels** in the given sentence:

"This is my first Degree in Saveetha School of Engineering."

ALGORITHM

1. Start the Prolog program.
2. Define the vowels as a, e, i, o, u.
3. Store the given sentence as a list of characters.
4. Check each character in the sentence.
5. If the character is a vowel, increment the count by 1.
6. If the character is not a vowel, continue with the next character.
7. Repeat until all characters are checked.
8. Display the total number of vowels.

Input:

```
count_vowels([], 0).

count_vowels([H|T], Count) :-
    member(H, "aeiou"),
    count_vowels(T, C),
    Count is C + 1.

count_vowels([H|T], Count) :-
    \+ member(H, "aeiou"),
    count_vowels(T, Count).

solve :-
    String = "this is my first degree in saveetha school of engineering.",
    count_vowels(String, Count),
    write('Number of vowels = '),
    write(Count),
    nl.
```

OUTPUT:

```
| ?- solve.
uncaught exception: error(existence_error(procedure,string_chars/2),solve/0)
| ?- ['C:/Users/lenovo/OneDrive/Documents/vowel(a).4.pl'].
compiling C:/Users/lenovo/OneDrive/Documents/vowel(a).4.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/vowel(a).4.pl compiled, 16 lines read - 4838 bytes written, 9 ms

(15 ms) yes
| ?- solve.
Number of vowels = 19

true ?
```

Result:

The Prolog program was successfully executed to count the number of vowels in the given sentence.